Advanced topics CBL

Password Game (.06)
Fenna Thijs & Feyza Sayar

Unit testing

Source: Unit Testing - GeeksforGeeks

Unit testing tests different components of a project in isolation to verify every part of the project runs as expected. It usually uses eight different steps to fully check the project. It can be performed manually or automatic. Since our project isn't too large, we tested our project manually, without it taking up too much valuable time.
Unit testing has two distinct components. First, all different parts of the project (units) should be tested independently. Secondly, the testing follows a so-called AAA pattern: Arrange, Act, Assert. This is basically a begin, middle, end, and a good fairytale ending is preferred.

The checks that we mostly looked at were checks of the type Logic Checks and Boundary Checks. The Logic Checks were used to make sure that the logical structure is correct, and the methods call each other in the expected way. Because we use multiple different booleans and methods that all call each other to check the user's input, it is necessary to make sure this all works in the right way.
Additionally, the Boundary Checks is what our game is also based upon, checking the input and giving feedback. We tested a big variety of inputs and asked others to test our game, to see what creative ways they could come up with to get around the rules of the game. We were able to use their input and questions to change the wording of rules or the methods themselves.

There also exist three different types of testing a project;
Black Box Testing is used to test the project from the user's perspective, without taking the code behind the scenes into account. We did this by just opening and running the program time and time again, and asking the same civilians as mentioned before to try our game. Because they didn't know our code at all, it was a very effective method to see what worked best.
Next, *White Box Testing* does the complete opposite. It tests the internal code and structure to make sure the code is logical and functional. In the start, we did this by separately testing the GUI and the methods responsible for checking the password

rules. Among other things, we were able to use System.out.print to see the outputs of our test cases, before connecting the outputs to the labels used in the GUI.
And lastly, *Gray Box Testing* mixes these tests to come to a middle ground. We implemented this while running our game and testing different inputs. Because we built the code, we had that extra knowledge of the core of the game to help us test the project in the best way.

There's multiple characteristics that 'make or break' unit testing. To productively test our project, we had to make sure the testing was efficient, repeatable, and focused on one singular part (in our case focused on either rules or useability of the GUI).
There are a few limitations to this way of testing though. The testing is usually focused on specific components of the project and can be less useful when verifying the overall project. It can also be overwhelming to fully unit test a larger project, because that requires more effort for all different methods, and a small mistake made by the programmer of the test can kill the efficiency of the testing.

(Some automated testing tools we could've used if our project became too big to fully manually check are JUnit or TestNG)

Git (version control)

Because we kept adding new rules and thus methods to our game, we implemented git to keep track of all changes we made. At first, when ideas are still in the making, different branches can really help. The use of this came back when we were experimenting with adding new rules and using different expressions to check the input of the user. A branch can easily be stranded if it turns out that it wasn't the right fit after all

But where version control made the most difference is in the comments and descriptions we were able to add to all the changes in the main branch. We both have filled schedules and we're usually working on code in the bits of free time we can find, and git made adding code a lot easier. Before, when we were still working on the first assignments and exercises, we struggled to share code and keep our code the same. Being able to add our files in the same project online and adding comments worked great to combat this.